RBAC Bot for Payment Hub

Architecture, Logic, Engine Design, Technology Stack, Features, Benefits and Roadmap

Version 1.0

Prepared for the current offline Windows-server implementation of the RBAC end-user support application.

Document Purpose

This document explains the complete design of the current **rbac_bot** application that was built to assist Payment Hub end users with role-related issues. It covers the business problem, solution approach, architecture, runtime flow, engines, files, technologies, deployment model, current limitations, and future enhancement roadmap.

Scope

- Covers the chatbot application currently running locally on a Windows server.
- Covers the current implementation based on Python, static HTML/CSS/JavaScript, and a lightweight local HTTP server.
- Covers only the current offline solution and the immediate planned direction for strengthening the role knowledge base and UI.

1. Executive Summary

The RBAC Bot is a lightweight, offline-first support assistant for Payment Hub role and access queries. It helps end users and support staff resolve common access problems without relying on external APIs or cloud-hosted AI services. The current solution is deliberately simple and enterprise-safe: it runs on a Windows server, uses only Python standard libraries for serving and orchestration, and stores its knowledge in structured local JSON files.

The application answers questions such as required roles for investigations, reasons why a tile or menu is not visible, which workflow role is required for a queue, and how Concourse notification mailboxes should be configured. Internally, it combines text normalisation, basic intent classification, fuzzy matching across a curated knowledge base, troubleshooting guidance, and interaction logging.

From a product perspective, the RBAC Bot is not a generic chatbot. It is a focused operational assistant for Payment Hub and Concourse role issues. Its value lies in reducing repeated support effort, answering predictable questions faster, and giving users a simple front door to role-related guidance while remaining consistent with internal role definitions and operational practices.

2. Problem Statement

Payment Hub and Concourse users frequently experience role-related issues such as missing tiles, hidden menu options, incorrect workflow access, notification mailboxes not receiving Concourse emails, and incomplete role assignments during onboarding or maintenance. These issues often result in support tickets, operational delays, repeated manual checks in ITIM/ISIM, and confusion across client onboarding, Call Direct, and support teams.

The problem is not only technical; it is also knowledge-distribution related. The required information exists across role exports, onboarding checklists, working practices, and operational experience, but it is not always easy for end users or first-line support staff to navigate quickly. The RBAC Bot addresses that gap by providing a focused, structured, searchable layer over the role knowledge.

3. Solution Overview

The current solution consists of five layers:

- A local static web interface (**rbac_ui.html**) for entering queries and displaying responses.
- A lightweight local HTTP server (**server.py**) listening on localhost and forwarding requests to the Python application.
- An application controller (**main.py**) that orchestrates request processing.
- A modular engine layer under the **engine** folder for normalisation, intent classification, Q&A; matching, troubleshooting matching, and interaction logging.
- A local knowledge layer under the **knowledge** folder containing the role Q&A; dataset, troubleshooting patterns, and synonyms used to improve matching.

The solution is intentionally offline and dependency-light. No Flask, database server, vector store, or external API is required to run the current version. This keeps deployment simple in a locked-down internal environment and preserves a clean upgrade path for later integration into a more formal enterprise-hosted architecture.

4. High-Level Architecture

The current runtime architecture is shown below.

```
User Browser → rbac_ui.html → JavaScript fetch request to http://localhost:5050/query → server.py → subprocess call to python main.py <query> → main.py → engine modules → local JSON knowledge base → response returned to browser → interaction logged to file.
```

This architecture keeps the UI and the query-processing logic separated. The browser only handles presentation and request submission. The actual business logic remains in Python and can later be reused behind a different UI or service layer.

5. Folder Structure

The application is organised into a simple folder structure designed for clarity and maintainability.

```
D:\Chatbot\rbac_bot\
■■■ main.py
■■■ server.py
■■■ rbac_ui.html
■■■ engine\
■ ■■■ __init__.py
■ ■■■ text_normalizer.py
■ ■■■ intent_engine.py
■ ■■■ qa_engine.py
■ ■■■ rule_engine.py
■ ■■■ logger.py
■■■ knowledge\
■ ■■■ qa.json
■ ■■■ troubleshooting.json
■ ■■■ synonyms.json
■■■ logs\
■■■ chatbot.log
```

6. Detailed Runtime Logic

6.1 Browser Layer

The web UI is a static HTML page containing the query input box, submit button, tabs / visual sections, and a response panel. JavaScript captures the query and sends it to the local backend using an HTTP GET request. It also updates the response area with the returned text. The UI is currently presentation-only; it does not perform business logic.

6.2 Local Server Layer

server.py uses Python's built-in **http.server** implementation to expose a local endpoint. When the UI submits a query, the server extracts the query string and executes **main.py** as a subprocess. This keeps the server very lightweight and avoids introducing a larger web framework at this stage.

The server captures the standard output of **main.py** and returns it to the browser as text. If the subprocess fails, the error text is returned, which helps during development and debugging.

6.3 Application Controller

main.py is the main orchestration layer. It supports two operating modes:

- Interactive CLI mode: when executed without arguments, the script runs as a console chatbot loop.
- UI / API mode: when a query is passed as a command-line argument, it processes a single query and writes the response to standard output for the browser to consume.

The primary function in the file is **process_query(query)**. This function performs the following sequence:

- Classify the query intent using the intent engine.
- If the intent is troubleshooting, attempt to match the query against known issue patterns and return a list of checks if found.
- Otherwise attempt to find an answer in the Q&A; engine.
- If no Q&A; match exists, return a standard 'No match found' response.
- Write the query, intent, and response to the log file.

7. Engine Design

The engine layer contains the reusable application logic. Each module has a single responsibility.

7.1 text_normalizer.py

This module standardises incoming text before matching. It performs three key tasks:

- Lower-case conversion.
- Removal of punctuation and normalisation of whitespace.
- Replacement of recognised variants using the synonym dictionary.

For example, a phrase such as ‘not able to raise case’ can be normalised into terms closer to ‘cannot create dispute’. This increases match quality without requiring exact phrasing.

7.2 intent_engine.py

This module classifies the user query into one of several simple intents. The current intents are:

- troubleshooting
- access_requirement
- role_explanation
- how_to
- general

The engine uses a straightforward keyword scoring model. It normalises the query first, then checks whether configured intent keywords appear in the text. The intent with the highest score is chosen. If no keywords match, the intent falls back to **general**.

This design is deliberately simple. It is transparent, easy to maintain, and sufficient for the current use case of role and access support. It also reduces the risk of surprising behaviour compared with more opaque classification strategies.

7.3 qa_engine.py

This engine is responsible for normal information retrieval. On startup, it loads **qa.json** and builds a **SEARCH_INDEX** consisting of:

- The primary question text from each record.
- Any query synonyms associated with that record.

Each entry is normalised using the text normaliser, then stored as a pair of **normalised_text** and the original Q&A; item. At runtime, the engine normalises the incoming query, compares it with the candidate texts using Python’s **difflib.get_close_matches**, and returns the answer from the closest match when the similarity threshold is high enough.

This fuzzy-matching design means users do not need to type the exact wording stored in the Q&A; file. They can ask naturally and still get a useful result.

7.4 rule_engine.py

This engine is the troubleshooting equivalent of the Q&A; engine. It loads **troubleshooting.json**, creates a searchable index using issue names and synonyms, and again uses **get_close_matches** to find the best issue match. If a match is found, it returns a list of troubleshooting checks instead of a single answer string.

The rule engine is only invoked when the classified intent is troubleshooting. This is important because it prevents informational questions from being incorrectly answered with diagnostic steps.

7.5 logger.py

This module writes interactions into **logs\chatbot.log**. Each entry captures the timestamp, original query, classified intent, and final response. Logging supports debugging, operational insight, and future knowledge-base improvement because repeated unanswered questions can be analysed and added back into the datasets.

8. Knowledge Layer Design

8.1 qa.json

This file is the main knowledge base for informational queries. Each record contains:

- question: the canonical question.
- synonyms: alternate ways users may ask the same thing.
- answer: the response text returned by the bot.

The Q&A; dataset is where domain expertise is encoded. It should contain answers about required Payment Hub roles, Concourse roles, contact groups, workflow constraints, common onboarding prerequisites, and role-specific feature visibility questions.

8.2 troubleshooting.json

This file stores issue-oriented patterns and the checks that should be returned when those issues are raised. It works best for short operational questions such as ‘cannot create dispute’ or ‘investigation option missing’.

8.3 synonyms.json

This file improves matching quality by defining canonical words and their variants. It is a low-effort, high-value way to make the bot feel smarter without changing engine code.

A key design principle is that the quality of bot responses depends heavily on the quality of the JSON knowledge files. The engines are intentionally simple, so dataset quality matters more than algorithmic complexity in the current version.

9. Technology Stack

Layer	Technology / Library	Purpose
Runtime	Python 3	Core application runtime on Windows server
UI	HTML, CSS, JavaScript	Static local browser interface
HTTP Server	http.server (Python standard library)	Local query endpoint
Process Execution	subprocess (Python standard library)	Executes main.py from server.py
Text Processing	json, re, os (Python standard library)	Load datasets and normalise text
Matching	difflib.get_close_matches	Fuzzy matching of queries to records
Logging	datetime + file append	Local interaction logging
Deployment	Windows Server	Local execution environment

The deliberate choice here is a minimal technology stack with no third-party Python web framework. This reduces installation friction and keeps the app workable in restricted enterprise environments. It also means the current solution is easy to understand and troubleshoot.

10. Request Processing Sequence

A typical request follows this sequence:

```
1. The user enters a query in the browser and submits the form.
2. JavaScript sends a GET request to http://localhost:5050/query?q=....
3. server.py extracts the query string.
4. server.py launches python main.py "<query>" as a subprocess.
5. main.py calls process_query().
6. process_query() calls classify_intent().
7. If the intent is troubleshooting, diagnose() is called; otherwise get_answer() is called.
8. The final response is written to standard output and logged.
9. server.py returns that output to the browser.
10. The browser updates the response panel.
```

11. Current UI Design

The current UI is a static browser page designed for low complexity and fast local testing. It includes a branded header, tabs / category buttons, a single query input, a submit button, and a response panel. The interface is intentionally simple so that the focus remains on the correctness of answers and the usability of the query workflow.

The UI colour treatment has started moving toward the Payments Hub look-and-feel, but the current implementation remains a local prototype rather than a portal-grade application. The browser page is suitable for demonstrating the concept and testing knowledge-base quality, but it is not yet a final hosted enterprise user experience.

12. Features Delivered in the Current Version

- Offline execution on a Windows server with no external API dependency.
- Role-question answering from a curated Payment Hub / Concourse knowledge base.
- Troubleshooting guidance for common role and visibility issues.
- Synonym-based text normalisation to support more natural user queries.
- Intent classification to separate diagnostics from informational questions.
- Fuzzy matching so users do not need exact wording.
- Local web UI for easier interaction than pure CLI mode.
- Interaction logging for future analysis and dataset improvement.

13. Benefits of the Current Solution

13.1 Operational Benefits

- Reduces repeated support effort for common role and access questions.
- Speeds up first response to end-user queries.
- Provides a consistent answer layer instead of relying on memory or ad hoc guidance.
- Creates a repeatable path for knowledge capture and reuse.

13.2 Technical Benefits

- Runs with minimal dependencies in a restricted environment.
- Modular design makes individual engines easy to replace or improve later.
- UI layer is separate from logic layer, allowing future migration to a new front end.
- Knowledge is stored as data, not hardcoded into the application flow.

13.3 Governance Benefits

- Local logs support auditability and problem analysis.
- Role guidance is based on curated datasets rather than freeform generation.
- The offline design aligns well with sensitive operational environments.

14. Limitations of the Current Version

- Knowledge quality depends on the completeness and consistency of the JSON datasets.
- Intent classification is keyword-based and therefore simple rather than robust.

- The local HTTP server currently shells out to a subprocess for every query, which is acceptable for a small prototype but not ideal for high-volume production usage.
- The current UI is a demonstrator interface rather than a portal-hosted enterprise application.
- There is no direct integration yet with ITIM, Payment Hub backend systems, or live role validation sources.
- Role validation UI and workflow-specific action routing have not yet been finalised in the browser interface.

15. Recommended Immediate Improvements

- Expand and continuously refine **qa.json** with real user phrases and support scenarios.
- Refine **synonyms.json** to improve text normalisation and fuzzy-match quality.
- Add a dedicated role validation mode and surface it in the UI.
- Make the Access / Troubleshooting / Validation tabs functionally distinct in the browser.
- Improve logging analytics to identify top failed queries and top repeated questions.
- Introduce a service-style architecture inside Python so that **server.py** can call functions directly rather than spawning a subprocess for each request.

16. Target Future Architecture

The likely future state is a hosted enterprise application integrated into the Payments Hub environment rather than a local HTML file and localhost server. In that future state, the current modular Python logic could either be preserved behind an internal service layer or partially reimplemented in a platform language compatible with the target hosting environment. The current prototype is a valuable stepping stone because it has already separated UI, engines, and knowledge.

A stronger hosted version would likely contain:

- Portal-hosted UI aligned with Payments Hub styling and navigation.
- A service/API layer instead of local subprocess execution.
- A richer role-knowledge store with version control and ownership.
- Optional direct integration with user-management systems for validation or guided request creation.
- Operational analytics for trend detection and support reduction reporting.

17. Roadmap

17.1 Phase 1 – Current Prototype

- Offline bot with web UI.
- Q&A;, troubleshooting, normalisation, intent classification, logging.

17.2 Phase 2 – Knowledge and UX Strengthening

- Expand RBAC datasets using real support scenarios.
- Improve synonyms and classification rules.
- Make tabs functional and add structured output cards.
- Add role validation and better troubleshooting workflows.

17.3 Phase 3 – Application Hardening

- Replace subprocess calls with direct in-process service handling.

- Add stronger error handling and admin diagnostics.
- Introduce packaging / release process for server deployment.

17.4 Phase 4 – Enterprise Integration

- Host within a Payments Hub-aligned environment.
- Introduce authenticated enterprise access.
- Enable optional integrations with live role or user-management sources.
- Add reporting on support reduction and commonly requested role topics.

18. Conclusion

The current **rbac_bot** is a focused operational assistant for Payment Hub role issues. Its strength is not in advanced AI or flashy technology, but in a clean, offline architecture built around modular Python engines and a curated knowledge base. Because the application already separates matching logic, intent classification, normalisation, troubleshooting, and UI, it provides a strong foundation for controlled growth. The next major value comes from enriching the knowledge layer and tightening the user experience while preserving the low-risk, enterprise-friendly deployment model.

Appendix A – Key Design Assumptions

- The current document reflects the implementation built during this working session and the file/module structure used in the local Windows server setup.
- The document is written for the Payment Hub role-assist use case, not as a generic chatbot design.
- Where future-state architecture is mentioned, it is described as a logical recommendation rather than an implemented feature.